CL1002 – Programming Fundamentals

Finals Practice Paper: Loops, Nested Loops, 1D Arrays, Sorting and Strings (C++)

16 questions | graded Easy → Very Hard | FAST-NUCES BCS Semester 1

How to use this paper

- Attempt **on paper first** (exams are handwritten / limited IDE time), then type it in and test with your own extra inputs, especially edge cases.
- Suggested time: Easy 10 min each, Medium 20 min, Hard 30 min, Very Hard 40 min. If stuck for 10 minutes, read the hint in the Appendix, then retry.
- Rules (same as your labs): no user-defined functions, no vectors, no *sort()*/*reverse()*/*find()* or other built-in algorithms, no 2D arrays. Strings are **null-terminated char arrays** read with *cin.getline()*. You may use *strlen()* only; do all other string work with loops.
- Validate every input exactly as stated (re-ask the same item until valid). Never touch an index outside 0 to size-1.
- Sample outputs are illustrative; your program must work for all valid inputs.

Classic exam traps to watch for

- Off-by-one: *i* <= *SIZE* instead of *i* < *SIZE*; *arr[j+1]* when j reaches the last index.
- = instead of ==; integer division (*sum* / *n*) when a decimal average is required.
- Forgetting to initialise sum/count/max (and initialising max to 0 when values can be negative).
- Mixing *cin* >> and *getline()* without *cin.ignore()*; forgetting the '\0' terminator on a char array you built yourself.
- Updating a flag or counter inside the wrong loop; printing inside the loop that should only compute.

SECTION A – EASY (warm-up, Q1 to Q4)

Question 1: Even/Odd Statistics [5 marks | Easy]

Declare an array of size 10 and input ten integers from 1 to 100 (re-ask an invalid value). Using loops, display: the count of even values, the count of odd values, the sum of even values, the **decimal** average of the even values (2 decimal places), and the largest odd value. If there is no even value, print "No even values" instead of the average; if there is no odd value, print "No odd values".

Key requirements

- Largest-odd must not start from 0 blindly: handle the "no odd value" case with a flag or counter.
- Avoid division by zero when there are no even values.

Sample run

```
Enter 10 values (1 to 100): 12 7 45 30 9 88 21 64 5 17
Even count: 4
Odd count: 6
Sum of even values: 194
Average of even values: 48.50
Largest odd value: 45
```

Question 2: Hollow Diamond [6 marks | Easy]

Input an **odd** number n from 3 to 15 (re-ask until valid). Using nested loops, print a hollow diamond of stars that has n rows (top half grows, bottom half shrinks). The widest row has n characters.

Key requirements

- Use only nested for-loops and cout of single characters (' ' and '*').
- Do not hard-code the output for n = 5; it must work for every odd n in range.

Sample run

```
Enter odd n (3 to 15): 5
  *
```

```
 * *
*   *
 * *
  *
```

Question 3: Dry Run (Output Prediction) [6 marks | Easy]

Write the exact output of the program below. Show a trace table for **s** and **a[i]** in Part 1, and list every value printed in Part 2. Mark where a line break or '|' appears.

```
#include <iostream>
using namespace std;
int main() {
    int a[6] = {4, 7, 2, 9, 5, 1};
    int s = 0;
    for (int i = 0; i < 6; i++) {          // Part 1
        if (a[i] % 2 == 0) s += a[i];
        else               s -= a[i];
        a[i] = s;
    }
    for (int i = 0; i < 6; i++) cout << a[i] << " ";
    cout << endl;

    int count = 0;                          // Part 2
    for (int i = 1; i <= 4; i++) {
        for (int j = i; j >= 1; j--) {
            if ((i + j) % 3 == 0) continue;
            cout << j;
            count++;
        }
        cout << "|";
    }
    cout << endl << count << endl;

    int x = 5, y = 2;                       // Part 3
    double r = x / y;
    cout << r << " " << x % y << " " << (x / y) * 2.0 << " " << x / 2.0 << endl;
    return 0;
}
```

Question 4: Find and Fix the Bugs [6 marks | Easy]

The program should read 5 marks (0 to 100), print the **decimal** average, the highest mark and how many marks are above the average. It contains **six** errors (logical, syntax or runtime-risk). Identify each, state the line, and write the corrected line.

```
 1  #include <iostream>
 2  using namespace std;
 3  int main() {
 4      const int SIZE = 5;
 5      int marks[SIZE], total, highest = 0, above = 0;
 6      for (int i = 0; i <= SIZE; i++) {
 7          cout << "Enter mark " << i + 1 << ": ";
 8          cin >> marks[i];
 9          while (marks[i] < 0 && marks[i] > 100) {
10              cout << "Invalid. Enter again: ";
11              cin >> marks[i];
12          }
13          total = total + marks[i];
14      }
15      double average = total / SIZE;
16      for (int i = 0; i < SIZE; i++) {
17          if (marks[i] > highest)
18              highest = marks[i];
19          if (marks[i] = average)
20              above++;
21      }
22      cout << "Average: " << average << endl
23      cout << "Highest: " << highest << " Above: " << above << endl;
24      return 0;
25  }
```

SECTION B – MEDIUM (core skills, Q5 to Q9)

Question 5: Second Largest Distinct Value [8 marks | Medium]

Input eight integers from 1 to 99. **Without sorting**, find the largest value and the **second largest distinct** value (a value equal to the largest does not count), and display the index of the **first** occurrence of the second largest. If every value is the same, print "No second largest exists".

Key requirements

- Single pass with two tracking variables (plus index) is expected; no sorting, no second array.
- Test with: all equal values; largest repeated many times; second largest appearing last.

Sample run

```
Enter 8 values (1 to 99): 25 40 40 12 33 40 7 33
Largest: 40
Second largest distinct: 33 (first at index 4)
```

Question 6: Palindrome Check and Letter Frequency [9 marks | Medium]

Read a sentence (max 80 characters) with *cin.getline()*. (a) Decide whether it is a palindrome **ignoring spaces, punctuation and letter case**, using two indexes that move towards each other (skip non-letters). (b) Display every distinct letter once (case-insensitive) with its frequency, in order of first appearance. Use an earlier-occurrence check with nested loops; **do not use a frequency array** and do not copy the string.

Key requirements

- Do not modify the original string permanently when comparing (use tolower() from <cctype> or manual +32 logic).
- Works for single-letter and empty-letter strings as well (e.g. "!!!" has no letters).

Sample run

```
Enter a sentence: A man a plan a canal Panama
Palindrome: Yes
a -> 10
m -> 2
n -> 4
p -> 2
l -> 2
c -> 1
```

Question 7: Rotate Right by k (In-Place) [8 marks | Medium]

Input nine integers, then a rotation count k from 1 to 8 (validate). Rotate the array to the **right** by k positions using only the same array and one temporary variable (no second array). Display the array before and after.

Key requirements

- Elements that fall off the right end re-enter at the left.
- Two accepted approaches: rotate by one position k times (nested loops), or reverse the whole array, then the first k, then the last n-k.

Sample run

```
Enter 9 values: 1 2 3 4 5 6 7 8 9
Enter k (1 to 8): 3
Before: 1 2 3 4 5 6 7 8 9
After : 7 8 9 1 2 3 4 5 6
```

Question 8: Merge Two Sorted Arrays [8 marks | Medium]

Declare arrays A (size 5) and B (size 6). Input values 1 to 50 into each so that each array is **non-decreasing**: if a value is smaller than the previous one, reject it and ask again. Merge them into a third array C (size 11) that is also sorted, using a single merge loop with three indexes (**do not sort C afterwards**). Finally, count how many distinct values of A also appear in B using nested loops.

Key requirements

- Handle the case where one array is exhausted before the other (leftover elements).
- Never read A[5] or B[6].

Sample run

```
Enter 5 sorted values for A: 2 5 9 14 20
Enter 6 sorted values for B: 1 5 8 14 22 30
Merged: 1 2 5 5 8 9 14 14 20 22 30
Common distinct values: 2
```

Question 9: Remove Duplicates In-Place [9 marks | Medium]

Input ten values from 1 to 20. Remove duplicates **in the same array**, keeping the first occurrence of every value and preserving order, by shifting the remaining elements left (no second array, no sorting). Display the new logical size, the compacted array, and how many elements were removed.

Key requirements

- Be careful: after shifting, the element now at index i must be checked again.
- Keep a variable for the current logical size; never read beyond it.

Sample run

```
Enter 10 values (1 to 20): 5 3 5 7 3 9 7 5 1 9
New size: 5
Array: 5 3 7 9 1
Removed: 5
```

SECTION C – HARD (Q10 to Q13)

Question 10: Bubble Sort (Descending) with Statistics [12 marks | Hard]

Input nine scores from 0 to 100. Using **Bubble Sort with a Boolean early-exit flag**, sort in **descending** order. After every pass print the array and whether a swap occurred. Report passes executed, total swaps and total comparisons. Then, from the sorted array only (no extra scans of the original data), display the **median** and the **mode** (smallest value if there is a tie in frequency) with its frequency. No *break*.

Key requirements

- The modal value should be found with nested loops or a run-length scan of the sorted array.
- Use strict comparison so equal values are never swapped.

Sample run

```
Enter 9 scores (0 to 100): 56 90 34 56 78 34 56 12 67
Original: 56 90 34 56 78 34 56 12 67
After pass 1: 90 56 56 78 34 56 34 67 12 | Swap: Yes
After pass 2: 90 56 78 56 56 34 67 34 12 | Swap: Yes
After pass 3: 90 78 56 56 56 67 34 34 12 | Swap: Yes
After pass 4: 90 78 56 56 67 56 34 34 12 | Swap: Yes
After pass 5: 90 78 56 67 56 56 34 34 12 | Swap: Yes
After pass 6: 90 78 67 56 56 56 34 34 12 | Swap: Yes
After pass 7: 90 78 67 56 56 56 34 34 12 | Swap: No
Passes: 7   Swaps: 13   Comparisons: 35
Median: 56
Mode: 56 (3 times)
```

Question 11: Reverse Every Word in a Sentence [12 marks | Hard]

Read a sentence (max 100 characters). Reverse the letters of **each word in place** while keeping the word order and every original space (words may be separated by more than one space). Then display the number of words and the longest word (original form) with its length; if two words tie, report the first.

Key requirements

- Use two indexes (start and end of each word) and a swap with a temporary char; no second string.
- Test: leading spaces, trailing spaces, multiple spaces, one-letter words, a sentence with one word.

Sample run

```
Enter a sentence: fast nuces is fun
Reversed words: tsaf secun si nuf
Word count: 4
Longest word: nuces (5 letters)
```

Question 12: Class Ranking with Tie-Breaking (Parallel Arrays) [13 marks | Hard]

Store seven student IDs (1000 to 9999, **unique**, reject duplicates) and their marks (0 to 50) in two parallel arrays. Using **Selection Sort**, sort by marks in **descending** order; when marks are equal, the **smaller ID comes first**. Keep the arrays synchronised. Then print a table with a **standard competition rank** (equal marks share a rank and the next rank skips, e.g. 1, 2, 2, 2, 5, 5, 7), the class average (2 decimals) and the number of students above the average.

Key requirements

- The tie-break must be part of the maximum search condition, not a separate sort.
- Do not create a third array; compute the rank with a running variable while printing.

Sample run

```
Enter ID and marks for student 1: 1204 38
 ... (7 students)
Rank  ID    Marks
1     1007  50
2     1099  45
2     1102  45
2     1415  45
5     1204  38
5     1350  38
7     1288  29
Class average: 41.43
Above average: 4
```

Question 13: Binary Search with Trace [12 marks | Hard]

Input ten integers (1 to 99) in **strictly increasing** order (re-ask any value that is not greater than the previous). Then repeatedly ask for a target; search using **binary search** (no *break*; control the loop with its condition). For every step print low, high, mid and the value at mid. Report whether the target was found, its index, the number of steps, and how many comparisons a linear search would have needed for the same target. Stop when the user enters 0.

Key requirements

- Use *mid = (low + high) / 2*; update with *mid + 1* / *mid - 1*. A wrong update causes an infinite loop.
- Handle a target smaller than the first or larger than the last element.

Sample run

```
Enter 10 increasing values: 3 8 12 15 21 27 34 40 46 59
Enter target (0 to stop): 34
Step 1: low=0 high=9 mid=4 value=21
Step 2: low=5 high=9 mid=7 value=40
Step 3: low=5 high=6 mid=5 value=27
Step 4: low=6 high=6 mid=6 value=34
Found 34 at index 6 in 4 step(s). Linear search needs 7 comparison(s).
Enter target (0 to stop): 10
Step 1: low=0 high=9 mid=4 value=21
Step 2: low=0 high=3 mid=1 value=8
Step 3: low=2 high=3 mid=2 value=12
10 not found after 3 step(s). Linear search needs 10 comparison(s).
Enter target (0 to stop): 0
```

SECTION D – VERY HARD (exam killers, Q14 to Q16)

Question 14: Run-Length Encoding and Decoding [15 marks | Very Hard]

Read a string of **lowercase letters only** (max 50 characters; re-ask if any other character appears). Build the **run-length encoded** string in a second char array: each run of the same letter becomes the letter followed by the run length (e.g. aaabccdddd → a3b1c2d4). Run lengths can exceed 9 (z12 for twelve z's), so you must convert the number into digit characters manually. Then **decode** the encoded array back into a third char array and verify that it equals the original (compare character by character). Display the encoded string, its length, whether it is shorter or longer than the original, and the verification result.

Key requirements

- Do not use to_string(), itoa(), sprintf(), atoi() or stoi(). Extract digits with % 10 and / 10 and reverse them yourself.
- Always terminate every char array you build with '\0'. Size the encoded array for the worst case (2 x length + 1).
- While decoding, a number may have more than one digit: accumulate with value = value * 10 + digit.

Sample run

```
Enter a lowercase string: aaabccdddd
Encoded: a3b1c2d4 (length 8)
Encoded string is shorter than the original (10 characters)
Decoded: aaabccdddd
Verification: Passed

Enter a lowercase string: zzzzzzzzzzzzab
Encoded: z12a1b1 (length 7)
Encoded string is shorter than the original (14 characters)
Decoded: zzzzzzzzzzzzab
Verification: Passed
```

Question 15: Maximum Sum Subarray and Zero-Sum Subarrays [15 marks | Very Hard]

Input ten integers from -50 to 50. Using **three nested loops** (start index, end index, and an inner loop that adds the elements), examine every contiguous subarray. Display the subarray with the **maximum sum** (its start index, end index, elements and sum; if there is a tie, choose the one with the smaller start index, then the shorter one) and the **number of contiguous subarrays whose sum is exactly zero**. Then print the subarray with the maximum sum as a bar-style row: for each element print '#' repeated by its absolute value divided by 5 (integer division).

Key requirements

- Initialise the best sum from the first subarray you examine (or from the lowest possible value), never from 0, because all values might be negative.
- Test a case where every number is negative; the answer is the single largest (least negative) element.

Sample run

```
Enter 10 values (-50 to 50): 4 -9 6 -2 8 -10 3 5 -1 2
Maximum sum: 12 (indexes 2 to 4): 6 -2 8
Zero-sum subarrays: 2
6  -> #
-2 ->
8  -> #
```

Question 16: Balanced Brackets Checker (Array as a Stack) [18 marks | Very Hard]

Read a string of up to 50 characters made only of ( ) [ ] { } (re-ask if any other character is present). Decide whether the brackets are balanced by simulating a **stack** using a char array and an integer *top* (push on every opening bracket; on a closing bracket the top must be its matching opening bracket, which you then pop). Print "Balanced" or the **first** error found, using 1-based positions: a mismatching closing bracket (show expected vs found), a closing bracket with an empty stack, or the number of unmatched opening brackets left at the end. Also display the maximum nesting depth reached (the largest value of top + 1 at any time).

Key requirements

- Never read stack[top] when top is -1 (empty stack): check that first.
- Stop checking after the first error without using break or goto (use a Boolean flag in the loop condition).
- Depth is tracked during the scan; do not rescan the string.

Sample run

```
Enter brackets: {[()]}
Balanced
Maximum depth: 3

Enter brackets: {[(])}
Not balanced: mismatch at position 4, expected ')' but found ']'

Enter brackets: )(
```

```
Not balanced: closing bracket ')' at position 1 has no matching opening bracket

Enter brackets: ((()
Not balanced: 2 unmatched opening bracket(s)
```

Marks distribution

Section	Questions	Marks
A - Easy	Q1 - Q4	23
B - Medium	Q5 - Q9	42
C - Hard	Q10 - Q13	49
D - Very Hard	Q14 - Q16	48
Total	**16 questions**	**162**

APPENDIX: ANSWER KEY AND HINTS (try first, then check!)

Question 3: Dry-run answers

```
Part 1 trace:  i=0: a[0]=4 even  -> s=4,   a[0]=4
               i=1: a[1]=7 odd   -> s=-3,  a[1]=-3
               i=2: a[2]=2 even  -> s=-1,  a[2]=-1
               i=3: a[3]=9 odd   -> s=-10, a[3]=-10
               i=4: a[4]=5 odd   -> s=-15, a[4]=-15
               i=5: a[5]=1 odd   -> s=-16, a[5]=-16
Output line 1: 4 -3 -1 -10 -15 -16
Part 2 output: 1|2|21|431|        (i=3,j=3 and i=4,j=2 etc. skipped when (i+j)%3==0)
Output line 3: 7                  (count of printed digits)
Part 3 output: 2 1 4 2.5          (x/y is integer division = 2, stored in double as 2)
```

Question 4: Bugs

- Line 5: *total* is not initialised → *int total = 0*.
- Line 6: *i* <= *SIZE* goes out of bounds → *i* < *SIZE*.
- Line 9: *&&* can never be true → use || (marks < 0 || marks > 100).
- Line 15: integer division loses the decimals → *(double)total / SIZE*.
- Line 19: = is assignment and the condition is wrong → *if (marks[i] > average)*.
- Line 22: missing semicolon after *endl*.

Hints for the harder questions (approach only)

- **Q5:** keep *largest* and *second* (start second at a sentinel like 0, since values are 1 to 99). If a[i] > largest: second = largest, largest = a[i]. Else if a[i] < largest and a[i] > second: update second.
- **Q6:** left = 0, right = length-1; move each side past non-letters; compare lowercase letters. For frequency: if the letter already appeared at an earlier index, skip it; otherwise count with an inner loop.
- **Q7:** reverse(0..n-1), reverse(0..k-1), reverse(k..n-1) with a swap loop written three times (no functions!), or the shift-by-one method inside a for-k loop.
- **Q8:** *while (i < 5 && j < 6)* pick the smaller; then two more while loops copy leftovers. For common distinct values, check that A[i] is not equal to any earlier A element before searching B.
- **Q9:** for i; for j = i+1 while j < size: if a[j]==a[i], shift everything after j left by one, size--, and do not increment j.
- **Q10:** run the flag loop as *while (swapped)*; in the sorted array a mode is the longest run of equal neighbours; median is a[SIZE/2] for odd SIZE.
- **Q11:** scan with an index; when you hit a non-space, find the word end, then swap inward with temp; continue from end+1. Word count = number of transitions from space to non-space.
- **Q12:** inside the inner loop use: *marks[j] > marks[best] || (marks[j] == marks[best] && id[j] < id[best])*. Rank: if marks equal to the previous student's marks keep the previous rank, else rank = position + 1.
- **Q13:** *while (low <= high && !found)*. Linear comparisons for a found target = index + 1; for a missing target = 10.
- **Q14:** encode: count run length with an inner while; write letter, then digits of the count (store digits in reverse in a small temporary char area or compute the digit count first). Decode: letter, then accumulate digits until the next letter.
- **Q15:** for start; for end >= start; sum = 0; for k = start..end sum += a[k]; update best with strict >, so earlier start / shorter length wins ties automatically.
- **Q16:** stack[top++] on opening; on closing, if top == 0 (empty) error; else compare stack[top-1] with the matching opener. Control the loop with *i < length && valid*.

Tip for the exam: write the loop skeleton first (input, process, output), then fill in conditions. Partial marks are given for correct validation, correct loop bounds and correct output formatting even when the main logic is not perfect.